

MATRIX: A Multi-Vendor Automation Testbed for Research in ICS Security

1st Kananelo Chabeli

Department of Electrical Engineering
University of Cape Town
Cape Town, South Africa
chbkan001@myuct.ac.za

2nd Daniel Ramotsoela

Department of Electrical Engineering
University of Cape Town
Cape Town, South Africa
daniel.ramotsoela@uct.ac.za

Abstract—

***Index Terms—*industrial control systems, multi-vendor testbed, Modbus TCP, OPC-UA, micro-segmentation, cyber-physical systems**

I. INTRODUCTION

Industrial Control Systems (ICS) form the backbone of critical infrastructure, from water treatment plants to power grids and manufacturing lines. As these systems increasingly adopt Ethernet-based communication and converge with enterprise information technology (IT) networks, they inherit a threat surface that was never part of their original design [1]. This convergence has made ICS cybersecurity an active research field, but that research is only as convincing as the testbed used to validate it.

Two broad testbed philosophies dominate the literature. The first relies on pure simulation, using tools such as MATLAB Simulink or discrete-event process models [2], [3]. Simulation is cheap and safe, but Morris et al. show that it fails to capture the true, complex interactions between real control hardware, firmware, and physical dynamics [1]. The second relies on full-scale national facilities, such as the Idaho National Laboratory testbed [4]. These offer high fidelity, but they are built for enterprise-level assessment and standards development, and are rarely made available for open-ended university research [1]. Between these two extremes sits a smaller group of academic, physical, commercial-off-the-shelf (COTS) testbeds, and it is here that MATRIX (Multi-Vendor Automation Testbed for Research in ICS Security) is positioned.

Even among physical academic testbeds, most are built around a single vendor and a single protocol. The Secure Water Treatment (SWaT) testbed, for instance, uses Allen-Bradley ControlLogix PLCs throughout, connected over a single EtherNet/IP and CIP stack [5]. This uniformity simplifies experimentation and produces clean, repeatable results, but it also means the platform cannot be used to study what happens when protocols and vendors differ within the same process, such as how a protocol gateway becomes an attack surface in its own right, or whether compromising one vendor's controller can be used to pivot into another vendor's control domain. A small number of recent testbeds

have started to close this gap. Sicard et al. combine Siemens and Schneider Electric hardware in a naval defence testbed, but each vendor is confined to its own domain loop, and the two fieldbuses involved, Modbus RTU and Profibus, are only bridged at the coordination-PLC level [6]. Calder et al. present SWaP, a low-cost water-process testbed that pairs a Siemens S7-1200 with an Allen-Bradley Micro820 and is, to our knowledge, the closest existing academic testbed to the one presented here [7]. In SWaP, however, a Raspberry Pi control unit writes sensor values directly into PLC memory rather than mediating between vendor protocols, and the two PLCs do not exchange process data with one another directly [7]. Separately, Kraust et al. review the ICS testbed literature and find that Modbus is heavily over-represented while OPC UA, a modern, vendor-neutral, service-oriented protocol, remains largely absent [8]. Purely virtual or containerised alternatives tend to trade protocol diversity away entirely in exchange for cost and reproducibility, typically supporting only Modbus TCP alongside IT-facing protocols such as MQTT or HTTP [9].

MATRIX is designed to close this gap directly. The testbed drives a single, shared Fischertechnik industrial process using three PLCs from three major automation vendors: a Siemens S7-1200, an Allen-Bradley (Rockwell) Micro820, and a Schneider Electric TM221CE16R. Each PLC governs a dedicated stage of the process and exchanges data with the others over Modbus TCP, so the process itself is genuinely distributed across vendors rather than merely co-located on a bench. A Raspberry Pi middleware layer then polls each PLC in its own native protocol, OPC UA for the S7-1200, EtherNet/IP for the Micro820, and Modbus TCP for the TM221CE16R, and re-publishes every value inside a single, modular OPC UA namespace. Supervisory control and historisation are handled by Ignition SCADA, a choice consistent with other recent hybrid ICS testbeds that pair a Raspberry Pi controller with an OPC UA-fed Ignition historian [10]. In effect, the middleware performs a form of protocol-level micro-segmentation: no external client ever needs to speak S7-comm, CIP, or a native Modbus register map directly, and every cross-vendor interaction is forced through a single, observable translation point. This mirrors, in a physical COTS setting, the intermediate mediation layer that Xie et al. use to

bridge OPC, Modbus, and Siemens S7 traffic in an otherwise virtual testbed [2], but extends the idea across three real vendors instead of one simulated process.

This heterogeneity is deliberate rather than incidental. It is what allows MATRIX to support research questions that a single-vendor or purely virtual testbed cannot easily raise, including how semantic mismatches between a Modbus register, a CIP tag, and an OPC UA node can be exploited to inject data that neither side flags as invalid, and how far a compromise of one vendor's PLC can travel before it is stopped. At the same time, because the underlying process is physical rather than simulated, MATRIX retains the timing, noise, and failure behaviour that Morris et al. and Gillen et al. identify as essential if security findings are meant to transfer to production environments [1], [11].

To summarise, the contributions of this paper are as follows:

- We present MATRIX, a physical, multi-vendor ICS testbed that unifies three PLC vendors and three industrial protocols behind a single OPC UA namespace, built entirely from low-cost, commercial-off-the-shelf hardware.
- We describe a Raspberry Pi middleware architecture that performs protocol translation and micro-segmentation, decoupling supervisory and research tooling from vendor-specific control protocols.
- We position MATRIX against existing physical, virtual, and hybrid ICS testbeds in the literature, and identify the multi-vendor, multi-protocol research questions it is uniquely placed to answer.

The remainder of this paper is organised as follows. Section V reviews related ICS and CPS testbeds in more detail. Section III describes the MATRIX architecture, including the physical process, the PLC layer, and the middleware. Section IV presents experimental results, and Section ?? concludes the paper and outlines future work.

II. RELATED WORK

ICS testbeds in the literature can be grouped along three broad axes: whether the platform is virtual, physical, or hybrid; how many industrial protocols and vendors it spans; and whether it uses a dedicated middleware layer to bring those protocols together. Kraust et al. survey a wide cross-section of existing testbeds along exactly these axes and find recurring weaknesses: a heavy skew towards Modbus, thin coverage of modern service-oriented protocols such as OPC UA, and limited attention to how IT and OT domains are actually bridged [8]. The remainder of this section reviews the literature against these axes and uses that review to motivate the design of MATRIX.

A. Virtual and Simulation-based Testbeds

The earliest response to the cost and risk of physical ICS experimentation was pure simulation. Reaves and Morris model remote terminal units, master terminal units, and physical process dynamics entirely in Python, and show that the resulting traffic and vulnerabilities are statistically close to a real water tank and gas pipeline [12]. Koganti et al. take

a similar approach for a power-distribution substation, using MATLAB/Simulink for the physical model and a Modbus PLC simulator for the controller, all running on virtual machines [3]. Xie et al. push this further with VTET, a virtualised Tennessee-Eastman process that can optionally hand control over to a physical Siemens PLC, bridging OPC, Modbus, and S7 traffic through an intermediate PC running an OPC server and an S7 proxy [2].

These platforms are inexpensive, safe, and easy to reset after a destructive experiment, which is precisely why Xie et al. use VTET to run reactor-explosion and productivity-loss attacks that would be irresponsible on real hardware [2]. However, none of them involve genuine third-party industrial hardware behaving under its own firmware, timing, and failure modes. Morris et al. make this limitation explicit: software simulation, however well tuned, cannot capture the true, complex interactions between real control hardware, its software stack, and the physical process it drives [1]. Xie et al.'s intermediate PC is nonetheless a useful precedent for MATRIX, because it shows that a single mediation point can sit between several industrial protocols without each downstream tool needing to speak all of them [2]. MATRIX adopts the same principle, but replaces a virtual process and a single simulated PLC with three physically distinct, commercially sourced controllers.

B. Large-scale and Academic Physical Testbeds

At the opposite end of the spectrum sit national-scale facilities, exemplified by the Idaho National Laboratory testbed, which combine full physical power grids and wireless infrastructure. Morris et al. note that such facilities are built for enterprise-level assessment and standards development, and are rarely made available for open-ended university research [1]. Their own Mississippi State University testbed was built specifically to fill that gap: it uses commercial-off-the-shelf PLCs and RTUs to drive real petrochemical, water, gas, manufacturing, and smart-grid processes, and it doubles as a platform for coursework and workforce development [1]. Candell et al. describe a similarly ambitious but differently organised NIST testbed, structured as independent enclaves, a continuous Tennessee-Eastman process and a discrete robotic assembly cell, sharing a common measurement and attack backbone, with the explicit goal of turning ICS security guidance such as NIST SP 800-82 and ISA/IEC-62443 into quantitative performance data [13]. Gillen et al. take realism further still: rather than approximating a generic process, they build a near-duplicate of the industrial cooling system for Oak Ridge National Laboratory's Summit supercomputer, using identical PLC hardware and configuration files, and validate that its network traffic and system state track the live production environment to within roughly one percent [11].

These testbeds establish that COTS hardware and a real physical process are what give a testbed evaluative value, a principle MATRIX shares. Their limitation, for the purposes of the present work, is protocol and vendor scope. The MSU testbed studies legacy serial and routable Ethernet protocols, but each process is generally driven by a single vendor's

equipment [1]; the NIST testbed's two enclaves each use one dominant protocol stack (DeviceNet and EtherNet/IP for the Tennessee-Eastman enclave, EtherCAT for the robotic cell) rather than several vendors sharing one process [13]; and the Oak Ridge testbed deliberately duplicates a single-vendor Allen-Bradley system to maximise behavioural fidelity to one specific production target [11]. None of them is designed to ask what happens when a single physical process is split across PLCs from different vendors that must talk to each other.

C. Homogeneous Physical Testbeds and the Cost of Uniformity

The Secure Water Treatment (SWaT) testbed is arguably the most widely used physical ICS testbed in the security literature. It replicates a six-stage municipal water-treatment process, but every stage is controlled by dual Allen-Bradley ControlLogix PLCs on a single EtherNet/IP and CIP stack [5]. This uniformity is a deliberate simplification: it produces clean, reproducible traffic and lets attacks and detectors be compared on equal footing. It also means that SWaT, by construction, cannot be used to study cross-vendor or cross-protocol phenomena such as a protocol gateway acting as an attack surface, or a compromise on one vendor's controller pivoting into another vendor's control domain, since only one vendor and one protocol are present [5].

Sicard et al.'s naval defence testbed goes one step further by combining Siemens and Schneider Electric hardware across four functional domain loops (direction, energy, artillery, and propulsion) of a scaled warship model [6]. This is a genuine multi-vendor platform, and its fieldbus mix, Modbus RTU on two domains and Profibus on the other two, adds protocol diversity that SWaT lacks. Crucially, however, each vendor and each protocol is confined to its own domain loop; two coordination PLCs bridge the domains only at a supervisory level, and there is no single, protocol-agnostic namespace that a downstream tool can query without knowing which domain, vendor, or fieldbus a given value came from [6]. The heterogeneity exists side by side rather than being unified.

D. Emerging Multi-vendor and Hybrid Testbeds

A small number of recent testbeds move closer to the design explored in this paper. Calder et al. present SWaP, a low-cost, modular water-processing testbed that pairs a Siemens S7-1200 with an Allen-Bradley Micro820, communicating over S7-comm and EtherNet/IP respectively, with a Raspberry Pi acting as a central control unit [7]. Of all the testbeds reviewed here, SWaP is the closest in spirit to MATRIX: it uses two of the same three PLC families, and it is explicit that heterogeneous, off-the-shelf hardware is preferable to a single-vendor stack for realistic security research [7]. The two designs diverge, however, in how the Raspberry Pi is used. In SWaP, sensor readings arrive over UDP from Arduino interface boards and are written directly into PLC memory by the control unit; the two PLCs do not exchange process data with one another, and there is no unified, protocol-independent namespace exposed to the rest of the system [7].

MATRIX instead uses the Raspberry Pi purely as a protocol-translation and aggregation layer sitting above three PLCs that already exchange process data directly over Modbus TCP, and every translated value is re-published inside a single OPC UA namespace rather than written into another device's memory space.

Gholap et al. take a different route to heterogeneity by combining a physical process with a virtual one. Their hybrid testbed pairs a Codesys-programmed Raspberry Pi, acting as a pseudo-PLC, with a Factory I/O virtual plant, and uses Ignition SCADA fed over OPC UA for historisation, the same SCADA and historian combination adopted in MATRIX [10]. This confirms that Ignition and OPC UA are a workable pairing for ICS research historisation. Its limitation is that the "PLC" in this design is a general-purpose computer emulating IEC 61131-3 behaviour rather than commercial PLC hardware, so the testbed cannot speak to how genuine vendor-specific firmware, timing, and protocol implementations behave, which is exactly the gap physical multi-vendor testbeds like SWaP and MATRIX are positioned to fill [7], [10].

Jorge et al. approach the same convergence problem from the opposite direction, with a fully containerised, Docker-based water-system testbed that integrates traditional ICS protocols with Industry 4.0 elements, using Modbus TCP for PLC control, MQTT for intermediate distribution, and HTTP for metering [9]. This is a low-cost, highly reproducible way to study IT/OT convergence and to generate labelled datasets, but the authors' own protocol choices illustrate the point made by Kraust et al.: Modbus, MQTT, and HTTP are covered, while OPC UA, EtherNet/IP, and vendor-specific protocols such as S7-comm are absent [8], [9]. Kraust et al. go on to argue that this absence matters because OPC UA's service-oriented, self-describing design is increasingly central to modern ICS deployments, and propose that a future OPC UA-centric testbed is needed to study it properly [8].

E. Positioning MATRIX

Table I summarises the comparison above along the dimensions that matter for the present work: whether the process is physical or virtual, how many PLC vendors and protocols are present, and whether a dedicated middleware layer unifies those protocols into a single namespace.

Three gaps emerge consistently from this review. First, testbeds that achieve high physical fidelity, such as the MSU testbed, SWaT, and the Oak Ridge cooling-system replica, generally do so by committing to a single vendor or a single protocol stack, which by construction rules out multi-vendor and multi-protocol research questions [1], [5], [11]. Second, the testbeds that do introduce multiple vendors or protocols, the naval testbed and SWaP, keep them side by side rather than unified: values remain tied to their originating domain, vendor, or memory address, so a researcher or a downstream detection tool still needs vendor-specific knowledge to use them [6], [7]. Third, OPC UA, the protocol best suited to acting as a vendor-neutral aggregation point, is either absent from these testbeds entirely or used only for historisation rather than as

TABLE I
COMPARISON OF MATRIX WITH REPRESENTATIVE ICS TESTBEDS IN THE LITERATURE

Testbed	Physical/Virtual	PLC Vendors	Protocols	Unified Namespace	Historian/SCADA
MSU Testbed [1]	Physical	Single per process	Serial + Ethernet	No	Commercial HMI
SWaT [5]	Physical	Single (Allen-Bradley)	EtherNet/IP, CIP	No	Commercial historian
Naval Testbed [6]	Physical	Siemens, Schneider	Modbus RTU, Profibus	No (per-domain)	Supervisory HMI
SWaP [7]	Physical	Siemens, Allen-Bradley	S7-comm, EtherNet/IP	No (direct memory write)	InfluxDB
Gholap et al. [10]	Hybrid (physical + Factory I/O)	Pseudo-PLC only	Modbus TCP, OPC UA	Partial (single controller)	Ignition SCADA
Jorge et al. [9]	Virtual (containerised)	N/A (software PLCs)	Modbus TCP, MQTT, HTTP	No	ELK stack
MATRIX (this work)	Physical	Siemens, Allen-Bradley, Schneider	OPC UA, EtherNet/IP, Modbus TCP	Yes (OPC UA)	Ignition SCADA

the substrate that unifies the platform, as confirmed by Kraust et al.'s literature review [8], [9].

MATRIX is designed to close all three gaps at once. It retains the physical fidelity of a real, shared industrial process, driven by genuine Siemens, Allen-Bradley, and Schneider Electric hardware. It goes beyond SWaP and the naval testbed by requiring its three PLCs to exchange process data with each other over Modbus TCP, so the process is truly distributed rather than merely multi-vendor. Finally, unlike every testbed reviewed here, it resolves the resulting protocol diversity, OPC UA, EtherNet/IP, and Modbus TCP, into a single modular OPC UA namespace at the middleware layer, so that vendor and protocol differences are handled once, centrally, rather than by every consumer of the data. This combination of physical realism, genuine multi-vendor process distribution, and protocol-level unification is, to our knowledge, not present together in any existing academic ICS testbed.

III. MATRIX ARCHITECTURE

This section describes the MATRIX testbed from the physical process upward: the shared industrial process itself, the three heterogeneous PLCs that control it, the network topology that separates control traffic from supervisory traffic, and the Raspberry Pi middleware that unifies the three PLC protocols into a single OPC UA namespace.

A. Physical Process

The shared physical process is a Fischertechnik Production Line 24V, a compact production-line model built for PLC training and control-system demonstration [14]. The line simulates a sequence of interlinked manufacturing steps. Workpieces are first loaded from a magazine onto a rotary indexing table using a pneumatic pusher. The indexing table has four stations and steps the workpieces around it one station at a time, presenting each workpiece to a drilling and welding station where it is processed. At the final station of the indexing table, a vacuum suction gripper lifts the finished workpiece off the table and places it onto a conveyor belt. The conveyor carries the workpiece to a palletizing station, where three completed workpieces are collected and then pushed together onto a pallet [14].

In MATRIX, this single physical process is split across three PLCs from three different vendors, so that no one controller owns the whole process:

- **Siemens S7-1200** controls the *turntable stage*: the magazine, the pneumatic pusher, the rotary indexing table, and

the drilling and welding station, up to the point where a finished workpiece is handed over to the transfer unit.

- **Allen-Bradley (Rockwell) Micro820** controls the *transfer unit*: the vacuum suction gripper that removes the finished workpiece from the turntable and places it on the conveyor belt.
- **Schneider Electric TM221CE16R** controls the *conveyor and the remainder of the process*: transport of the workpiece to the palletising station, collection of three workpieces, and the final push onto a pallet.

At Level 0 of the Purdue Enterprise Reference Architecture [15], the field devices are those supplied with the Fischertechnik model: mini motors, pneumatic cylinders driven through solenoid valves and a compressor, phototransistor and light-barrier sensing, and push-button inputs, giving eight digital inputs and ten digital outputs across the line. Splitting a single, mechanically continuous process across three vendors in this way is what allows MATRIX to study genuine cross-vendor process handover, rather than three unrelated processes running side by side, which is the arrangement used in the naval defence testbed's separate domain loops [6] and in the multi-enclave NIST testbed [13].

B. Network Topology and Purdue Model Mapping

MATRIX separates control traffic from supervisory traffic at the network layer, following the same Level 0/1 versus Level 2 separation used by SWaT [5] and by the naval defence testbed's distinct OT and IT switching infrastructure [6]. Two subnets are used:

- **192.168.50.0/24**, the control (Level 1) subnet, on which all three PLCs communicate directly with one another over Modbus TCP.
- **192.168.100.0/24**, the supervisory (Level 2) subnet, on which the Ignition SCADA host resides.

All devices connect through a single managed switch, a Teltonika Networks TSW212. Rather than mirroring the naval testbed's approach of two physically separate switches and two separate packet-capture hosts, one per domain [6], MATRIX configures a single port on the TSW212 as a Switched Port Analyser (SPAN) port, mirroring traffic from both subnets to a dedicated Monitoring Node running Wireshark. This follows the same principle as the measurement enclave in Candell et al.'s testbed, which likewise uses SPAN-port packet capture on a high-performance server to observe the whole platform from one vantage point rather than instrumenting each enclave separately [13]. Consolidating capture onto a single switch

and a single monitoring host keeps the physical footprint of MATRIX small without sacrificing visibility into either subnet, which matters for an academic testbed built on a limited budget [1].

The two subnets are not statically routed to one another. Instead, the Raspberry Pi middleware is dual-homed, with one network interface on each subnet, so that it is the sole point of passage between the control and supervisory domains. Every packet that crosses from Level 1 to Level 2 must therefore pass through the middleware’s protocol-translation logic rather than through a generic IP route. This is consistent with the intermediate mediation layer used by Xie et al. to bridge OPC, Modbus, and Siemens S7 traffic between a virtual process and its supervisory tooling [2], but here the mediation point also functions as the only network-layer bridge between the two subnets, giving MATRIX a natural point at which to enforce, log, or later restrict cross-domain traffic for zero-trust and micro-segmentation experiments.

A dedicated **AttackerNode** is also dual-homed across both subnets. In normal operation it serves as the engineering workstation used to program the three PLCs and to host the Ignition SCADA virtual machine, giving it legitimate reasons to reach both the control and supervisory subnets. This dual role mirrors two things in the literature at once: the dedicated attack platforms Candell et al. include alongside their measurement enclave for penetration testing and traffic manipulation [13], and the attacker container in Jorge et al.’s containerised testbed, which is deliberately attached to every internal network so that it can reach every component [9]. Giving the same node both an engineering role and an attack role is also realistic: a compromised engineering workstation with legitimate cross-domain access is one of the attacker models Mathur and Tippenhauer test against SWaT [5], and it reflects how many real ICS incidents begin, through a trusted host that already has the access an external attacker would otherwise have to work for.

C. Middleware: Protocol Translation and Namespace Unification

The Raspberry Pi middleware is implemented in Python around the `asyncio` event loop, with one coroutine-based client per PLC protocol: `asyncua` for the OPC UA connection to the S7-1200, `pycomm3` for the EtherNet/IP connection to the Micro820, and `pymodbus` for the Modbus TCP connection to the TM221CE16R. Running the three clients as concurrent asynchronous tasks rather than as three separate blocking processes allows the middleware to poll all three PLCs within a single, predictable cycle, without a slow response from one protocol stalling the others.

Each polled value is written into a single, modular OPC UA namespace exposed by an `asyncua` server running on the same Raspberry Pi. Structuring the namespace around named nodes, one per process variable, rather than around raw protocol addresses, follows the same principle behind the abstract “points” that Reaves and Morris use in their virtual testbed to let control logic ref-

erence variables by name instead of hardcoded memory addresses [12]. MATRIX applies this abstraction across three real, commercial protocols instead of one simulated one, so that Ignition SCADA, and any future research tooling, can query `Turntable.WorkpiecePresent` or `Conveyor.BeltRunning` without needing to know whether the underlying value came from S7-comm, CIP, or a Modbus holding register.

Ignition SCADA runs as a virtual machine on the supervisory subnet and connects to the Raspberry Pi purely as an OPC UA client, subscribing to the unified namespace for both its HMI displays and its historian. This mirrors the pairing of a Raspberry Pi controller with an OPC UA-fed Ignition historian used in a recent hybrid ICS testbed [10], and confirms that the same SCADA and historisation stack is workable when the OPC UA source is a real translation layer over physical, multi-vendor hardware rather than a single pseudo-PLC.

Finally, the three PLCs continue to exchange the time-critical handover signals for the process (for example, the turntable signalling that a workpiece is ready for the transfer unit) directly over Modbus TCP on the Level 1 subnet, without passing through the middleware. Keeping this exchange off the middleware’s polling path follows the same logic Morris et al. apply when validating their retrofit data logger: a device that observes or forwards control traffic must not sit inside the control loop’s timing budget, and their logger was accepted specifically because it added under 50 ms of latency against a baseline of roughly 35 ms, well inside the one to two second RTU polling period it was monitoring [1]. In MATRIX, the middleware only ever monitors and republishes state; it never sits between the two PLCs that must coordinate a physical handover in real time.

IV. EXPERIMENTS RESULTS

V. CONCLUSIONS

REFERENCES

- [1] T. Morris, A. Srivastava, B. Reaves, W. Gao, K. Pavurapu, and R. Reddi, “A control system testbed to validate critical infrastructure protection concepts,” *International Journal of Critical Infrastructure Protection*, vol. 4, no. 2, pp. 88–103, 2011.
- [2] Y. Xie, W. Wang, F. Wang, and R. Chang, “Vtet: A virtual industrial control system testbed for cyber security research,” in *2018 Third International Conference on Security of Smart Cities, Industrial Control System and Communications (SSIC)*. IEEE, 2018, pp. 1–7.
- [3] V. S. Koganti, M. Ashrafuzzaman, A. A. Jillepalli, and F. T. Sheldon, “A virtual testbed for security management of industrial control systems,” in *2017 12th International Conference on Malicious and Unwanted Software (MALWARE)*. IEEE, 2017, pp. 85–90.
- [4] K. Myers, “Idaho national laboratory (inl) microgrid testbeds,” in *Risk-Informed Methods and Applications in Nuclear and Energy Engineering*. Elsevier, 2024, pp. 259–262.
- [5] A. P. Mathur and N. O. Tippenhauer, “Swat: A water treatment testbed for research and training on ics security,” in *2016 international workshop on cyber-physical systems for smart water networks (CySWater)*. IEEE, 2016, pp. 31–36.
- [6] F. Sicard, E. Hotellier, and J. Francq, “An industrial control system physical testbed for naval defense cybersecurity research,” in *2022 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2022, pp. 413–422.
- [7] M. Calder, M. Ahmed, and S. Nagaraja, “Swap: A water process testbed for ics security research,” in *2023 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*. IEEE, 2023, pp. 1–4.

- [8] S. Kraust, P. Heller, and J. Mottok, "Concept for designing an ics testbed from a penetration testing perspective," in *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2025, pp. 561–568.
- [9] C. H. Jorge, L. Nacamura Jr, and A. C. B. K. Vendramin, "Containerized testbed architecture for cybersecurity data collection on malicious activities in industrial water systems," *Journal of the Brazilian Computer Society*, vol. 31, no. 1, pp. 612–627, 2025.
- [10] Y. Gholap, P. Yeola, and V. Singh, "On developing a hybrid cyber-physical testbed for cyber-secure industrial control system," in *2025 IEEE Latin Conference on IoT (LCIoT)*. IEEE, 2025, pp. 137–140.
- [11] R. E. Gillen, L. A. Anderson, C. Craig, J. Johnson, R. Anderson, A. Craig, and S. L. Scott, "Design and implementation of full-scale industrial control system test bed for assessing cyber-security defenses," in *2020 IEEE 21st International Symposium on "A World of Wireless, Mobile and Multimedia Networks"(WoWMoM)*. IEEE, 2020, pp. 341–346.
- [12] B. Reaves and T. Morris, "An open virtual testbed for industrial control system security research," *International Journal of Information Security*, vol. 11, no. 4, pp. 215–229, 2012.
- [13] R. Candell, T. Zimmerman, and K. Stouffer, "An industrial control system cybersecurity performance testbed," *National Institute of Standards and Technology. NISTIR*, vol. 8089, 2015.
- [14] fischertechnik GmbH. Production line 24v (model 571910). <https://www.fischertechnik.de/en/products/industry-and-universities/training-models/571910-production-line-24v>. Accessed: 2026-08-11.
- [15] T. J. Williams, "The purdue enterprise reference architecture," *Computers in industry*, vol. 24, no. 2-3, pp. 141–158, 1994.